

Algorithms Data Structures

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- o Explain Big O notation and how it helps in analyzing algorithms.
- o Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- o Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

3. Implementation:

- o Implement linear search and binary search algorithms.
- o Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- o Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Ans:

To implement Linear Search and Binary Search algorithms for searching products in an e-commerce platform and compare their time complexities. Searching is the process of finding a particular element in a collection of data.

Big O Notation

Big O notation is used to measure the efficiency of an algorithm by describing how its execution time grows as the input size increases.

Common complexities include:

- **$O(1)$** – Constant Time
- **$O(\log n)$** – Logarithmic Time
- **$O(n)$** – Linear Time
- **$O(n \log n)$** – Linearithmic Time
- **$O(n^2)$** – Quadratic Time

Search Operation Cases

Best Case

- The desired product is found at the first position.
- Linear Search: **$O(1)$**
- Binary Search: **$O(1)$**

Average Case

- The desired product is somewhere in the middle.
- Linear Search: **$O(n)$**
- Binary Search: **$O(\log n)$**

Worst Case

- The product is at the last position or not present.
- Linear Search: **$O(n)$**
- Binary Search: **$O(\log n)$**

Algorithm

Linear Search

1. Create a Product class.
2. Store products in an array.
3. Traverse the array one element at a time.

4. Compare each product ID with the search key.
5. Display the product if found.

Binary Search

1. Sort the product array according to Product ID.
2. Initialize low and high pointers.
3. Find the middle element.
4. Compare the middle Product ID with the search key.
5. Continue searching in the left or right half until found.

Program

Filename : Product.java

```
public class Product
{
    int productId;

    String productName;

    String category;

    public Product(int productId, String productName, String category)
    {
        this.productId = productId;

        this.productName = productName;

        this.category = category;
    }

    public void display()
    {
        System.out.println(productId + " " + productName + " " +
category);
    }
}
```

```
}  
  
}
```

Filename : SearchDemo.java

```
import java.util.Arrays;  
  
import java.util.Comparator;  
  
public class SearchDemo  
{  
  
    // Linear Search  
  
    public static Product linearSearch(Product[] products, int id)  
    {  
  
        for (Product product : products)  
  
            {  
  
                if (product.productId == id) {  
  
                    return product;  
  
                }  
  
            }  
  
        return null;  
  
    }  
  
  
    // Binary Search  
  
    public static Product binarySearch(Product[] products, int id)
```

```
{

    int low = 0;

    int high = products.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (products[mid].productId == id) {

            return products[mid];

        }

        if (products[mid].productId < id) {

            low = mid + 1;

        }

        else {

            high = mid - 1;

        }

    }

    return null;

}

public static void main(String[] args) {

    Product[] products = { new Product(105, "Laptop",
"Electronics"), new Product(101, "Shoes", "Fashion"), new Product(103,
"Phone", "Electronics"), new Product(102, "Watch", "Accessories"), new
Product(104, "Book", "Education") };

    System.out.println("Linear Search:");

    Product result = linearSearch(products, 103);

}
```

```

        if (result != null)

            result.display();

        else

            System.out.println("Product Not Found");

        Arrays.sort(products, Comparator.comparingInt(p ->
p.productId));

        System.out.println("\nBinary Search:");

        result = binarySearch(products, 103);

        if (result != null)

            result.display();

        else

            System.out.println("Product Not Found");

    }

}

```

Output

```

PS C:\Users\ashut\Downloads\Week 1\Algorithms_Data Structures\Exercise 2 E-commerce Platform Search Function> javac Product.java SearchDemo.java
PS C:\Users\ashut\Downloads\Week 1\Algorithms_Data Structures\Exercise 2 E-commerce Platform Search Function> java SearchDemo
Linear Search:
103 Phone Electronics

Binary Search:
103 Phone Electronics

```

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- o Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- o Create a method to calculate the future value using a recursive approach.

3. Implementation:

- o Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- o Discuss the time complexity of your recursive algorithm.
- o Explain how to optimize the recursive solution to avoid excessive computation.

Ans:

To implement a recursive algorithm in Java for predicting future financial values based on past growth rates and analyze its time complexity. Financial forecasting is the process of estimating future financial values based on historical data and growth trends.

Recursion

Recursion is a programming technique in which a method calls itself to solve a problem. It breaks a complex problem into smaller sub-problems until a base condition is reached.

Advantages of Recursion:

- Simplifies complex problems.
- Produces clean and readable code.
- Useful for mathematical calculations and divide-and-conquer algorithms.

Disadvantages:

- Increased memory usage due to function calls.
- Can lead to excessive computation if overlapping subproblems exist.

Algorithm

1. Read the present financial value.
2. Read the annual growth rate.
3. Read the number of years.
4. Create a recursive method **forecast()**.
5. If the number of years is zero, return the current value.
6. Otherwise, calculate the next year's value and call the same method recursively.
7. Display the predicted future value.

Program

Filename :FinancialForecast.java

```
import java.util.Scanner;

public class FinancialForecast
{
    // Recursive method

    public static double forecast(double currentValue, double growthRate,
int years)
```



```
{

    if (years == 0)

    {

        return currentValue;

    }

    return forecast(currentValue * (1 + growthRate / 100), growthRate,
years - 1);

}

public static void main(String[] args)

{

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter Current Value: ");

    double currentValue = sc.nextDouble();

    System.out.print("Enter Annual Growth Rate (%): ");

    double growthRate = sc.nextDouble();

    System.out.print("Enter Number of Years: ");

    int years = sc.nextInt();

    double futureValue = forecast(currentValue, growthRate, years);

    System.out.printf("Predicted Future Value: %.2f", futureValue);

    sc.close();

}
```

Output

```
PS C:\Users\ashut\Downloads\Week 1\Algorithms_Data Structures\Exercise 7 Financial Forecasting> javac FinancialForecast.java
PS C:\Users\ashut\Downloads\Week 1\Algorithms_Data Structures\Exercise 7 Financial Forecasting> java FinancialForecast
Enter Current Value: 10000
Enter Annual Growth Rate (%): 10
Enter Number of Years: 5
Predicted Future Value: 16105.10
```